Ders 17 Çalışma Özeti

Ders 17 Çalışma Özeti

Genel Konular

- Dersin tarihi 2 Aralık 2020 Çarşamba (9. hafta, vize haftasından sonraki ilk ders). Ders Zoom üzerinden yapıldı.
  - Asıl anlatım yaklaşık 00:17:30'da başlar. Önce vize hakkındaki geri bildirimler ve duyurular konuşulur.
  - 00:51:45 ile 01:02:10 arası ara.
  - Kayıt, sırt çantası (knapsack) probleminin formülü kurulmadan biter. Formül saat 15:00'teki uygulama oturumunda ele alınacaktı.
- Vize geri bildirimleri ve duyurular
- Dinamik programlama (dynamic programming)
  - Tanım, Richard Bellman
  - Optimizasyon ve kombinatorik problemler
  - Optimallik ilkesi (principle of optimality)
- Fibonacci sayıları: özyinelemeli çözüm ile dinamik programlama karşılaştırması
- Para sırası problemi (coin-row problem)
  - Rekürans bağıntısı, tablo, seçilen paraların geriye doğru bulunması
  - Zaman ve bellek karmaşıklığı
- Sırt çantası problemi (0/1 knapsack problem)
  - Kaba kuvvet (brute force) çözümün maliyeti
  - Dinamik programlama tablosu

Hocanın Özellikle Vurguladığı Kısımlar

- Ödev ve sınavlarda girdi/çıktı dosyası istenmesine hoca sıcak bakmadı.
  - Test durumlarını (case) düşünmek algoritma tasarımının parçasıdır. Hoca yaratıcılığı kısıtlamak istemiyor.
- "Optimizasyon problemi" bir algoritmayı hızlandırmak demek değildir.
  - Kısıtlı kaynakları, verilen kısıtlara uygun biçimde en iyi şekilde kullanma problemidir (en küçük ya da en büyük değeri bulma).
  - Örnekler: grafı en az renkle boyama, otobüs güzergâhı optimizasyonu, ders programı yerleştirme.
- Dinamik programlamanın zayıf noktası bellek kullanımıdır.
  - Problemi çözmeden önce gereken bellek (space complexity) hesaplanmalıdır.
- Bilgisayar "gözle bakarak" karar veremez. Her adımda bir formüle (rekürans bağıntısına) göre karar verilmelidir.
- Hoca derse katılımın düşük olmasından yine şikâyet etti.

Kısa Tekrar Notları

- Dinamik programlama: Örtüşen alt problemler çözülür, sonuçlar bellekte saklanır. Genelde aşağıdan yukarıya (bottom-up) ilerlenir.
  - 1950'lerde Richard Bellman ortaya attı. "Programming" burada "planlama" anlamındadır.
- Optimallik ilkesi: Optimal çözüm, alt problemlerin optimal çözümlerinden oluşur. Çoğu optimizasyon probleminde geçerlidir ama her problemde değil.
- n özellikten boş olmayan bütün alt kümeler: $2^n - 1$.

- Fibonacci: $F(n) = F(n-1) + F(n-2)$, $F(0) = 0$, $F(1) = 1$.
  - Düz özyineleme aynı değerleri tekrar tekrar hesaplar (üstel zaman).
  - Diziyle soldan sağa doldurmak $\Theta(n)$ zaman alır.
- Para sırası problemi: Yan yana iki para seçilmeden toplam en büyük değer.
  - $F(n) = \max\{c_n + F(n-2),\ F(n-1)\}$, $n > 1$; $F(0) = 0$, $F(1) = c_1$.
  - Zaman ve bellek: $\Theta(n)$.
- 0/1 sırt çantası: Kapasite W içinde toplam değeri en büyük eşya kümesi.
  - Kaba kuvvet: $2^n - 1$ kombinasyon.
  - Dinamik programlama tablosu: $(n+1)$ satır (eşyalar) × $(W+1)$ sütun (ağırlıklar). İlk satır ve ilk sütun 0. Cevap sağ alt köşededir.
  - $P(i, w) = \max\{P(i-1, w),\ v_i + P(i-1, w-w_i)\}$ ($w_i \leq w$ ise); $w_i > w$ ise $P(i, w) = P(i-1, w)$.

Detaylı Açıklamalar

Vize geri bildirimleri ve duyurular

- Vizenin ilk 15 dakikalık test kısmının yerinde olduğu söylendi. Bir sıralama algoritması tanıma sorusunda sayıların karmaşık olduğu eleştirildi.
  - Hoca bu soruda algoritmaya özel ipuçları olduğunu söyledi: Insertion sort'ta elemanın yerine yerleşmesine, Merge sort'ta ikiye bölünen yarıların sıralı olup olmadığına bakmak gibi.
- Klasik kısımda tam kod, kod ve analiz, hash kodu yazma soruları vardı. Bazı öğrenciler sürenin dar geldiğini söyledi. Hoca bunu Elif Hoca'ya aktaracağını ve son iki sınavda süreyi gözden geçireceklerini söyledi.
- İkinci vize yapılıp yapılmayacağı henüz belli değil.

Dinamik programlama

Bir problemin örtüşen alt problemleri çözülür ve sonuçları bellekte (bir dizi ya da tabloda) saklanır. Büyük problemin çözümüne bu saklanan değerlerden ulaşılır. Böylece özyinelemeli yaklaşımda tekrar tekrar çözülen alt problemler bir kez çözülür.

Genelde iki tür problemde kullanılır:

- Optimizasyon problemleri: kısıtlar altında en büyük ya da en küçük değeri bulma.
- Kombinatorik problemler: Örneğin bir sınıflandırma probleminde n özellikten en başarılı alt kümeyi seçmek. Boş küme hariç $2^n - 1$ seçenek vardır.

Optimallik ilkesi: Bir problemin optimal sonucuna, alt problemlerinin optimal sonuçlarını birleştirerek ulaşılabilir. Bazı istisnalar dışında geçerlidir.

Fibonacci sayıları

Dizi: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Özyinelemeli çözüm:

```
int fibonacci(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Sonuç doğrudur ama aynı alt problemler tekrar tekrar hesaplanır. Örneğin $F(5)$ hesaplanırken $F(3)$ iki kez, $F(2)$ üç kez hesaplanır.

Dinamik programlama ile bir dizi soldan sağa doldurulur: $F(0)$ ve $F(1)$ yazılır, sonraki her değer önceki iki değerin toplamıdır. Hiçbir değer tekrar hesaplanmaz.

Bedeli bellektir: Sonuçları saklamak için ek alan gerekir.

Para sırası problemi (coin-row problem)

Bir sırada paralar vardır. Yan yana iki para seçilemez. Amaç toplam değeri en büyük yapmaktır.

Örnek: $c = 5, 1, 2, 10, 6, 2$.

Her para için iki seçenek vardır:

- Parayı al: $c_n + F(n-2)$ (bir önceki para alınamaz).
- Parayı alma: $F(n-1)$.

$$F(n) = \max\{c_n + F(n-2),\ F(n-1)\}, \quad n > 1, \quad F(0) = 0,\ F(1) = c_1$$

İndis	0	1	2	3	4	5	6
c	-	5	1	2	10	6	2
F	0	5	5	7	15	15	17

- $F(2) = \max(1 + 0, 5) = 5$
- $F(3) = \max(2 + 5, 5) = 7$
- $F(4) = \max(10 + 5, 7) = 15$
- $F(5) = \max(6 + 7, 15) = 15$
- $F(6) = \max(2 + 15, 15) = 17$

Seçilen paraları bulma (geriye doğru): Sondan başlanır. $F(i) \neq F(i-1)$ ise i. para alınmıştır; bir önceki para alınamayacağı için $i-2$'ye atlanır. $F(i) = F(i-1)$ ise i. para alınmamıştır; $i-1$'e geçilir.

- $F(6) = 17 \neq F(5) = 15$: 6. para alındı, 4'e geç.
- $F(4) = 15 \neq F(3) = 7$: 4. para alındı, 2'ye geç.
- $F(2) = 5 = F(1) = 5$: 2. para alınmadı, 1'e geç.
- $F(1) = 5 \neq F(0) = 0$: 1. para alındı.

Seçilen paralar: 1., 4. ve 6. (5 + 10 + 2 = 17).

Karmaşıklık: Dizi bir kez taranır. Zaman ve bellek $\Theta(n)$'dir.

Bir öğrenci yalnız en büyük toplam isteniyorsa son üç değeri tutmanın yeteceğini söyledi. Hoca bunun doğru olduğunu, ama bu tür problemlerde genelde seçilen elemanların da istendiğini söyledi.

Sırt çantası problemi (0/1 knapsack)

Bir hırsızın çantasının taşıyabileceği bir ağırlık kapasitesi vardır. Evdeki eşyaların ağırlıkları ve değerleri bellidir. Kapasiteyi aşmadan toplam değeri en büyük eşya kümesi aranır.

Kaba kuvvet: Bütün alt kümeler denenir: $2^n - 1$ kombinasyon.

İlk örnek ($W = 5$):

Eşya	1	2	3	4	5
Ağırlık	2	1	3	4	5
Değer	10	20	15	50	90

Kapasiteye sığan kombinasyonlar: {1, 2} = 30, {2, 3} = 35, {1, 3} = 25, {2, 4} = 70, {5} = 90. En iyisi tek başına 5. eşyadır (90).

Dinamik programlama örneği ($W = 5$, tablo tahta karesinden):

Eşya	Ağırlık	Değer
1	2	12
2	1	10

Eşya	Ağırlık	Değer
3	3	20
4	2	15

Tablo: satırlar eşyalar (0-4), sütunlar ağırlık sınırı (0-5). 5 satır, 6 sütun gerekir. İlk satır (hiç eşya yok) ve ilk sütun (0 kilo) 0'dır.

$i \backslash w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

Cevap sağ alt köşededir: 37. Seçilen eşyalar 1, 2 ve 4'tür (12 + 10 + 15 = 37, ağırlık 2 + 1 + 2 = 5).

Her eşya için iki seçenek vardır: almak ya da almamak. Bu, rekürans bağıntısına götürür:

$$P(i,w) = \begin{cases} \max\{P(i-1,w),\ v_i + P(i-1,w-w_i)\} & w_i \le w \\ P(i-1,w) & w_i > w \end{cases}$$

$$P(0,w) = 0, \quad P(i,0) = 0$$

Derste formül tam yazılmadan kayıt bitti. Örnek olarak $P(1,3) = \max\{P(0,3),\ 12 + P(0,1)\} = 12$ gösterildi.

Tablo $(n+1)(W+1)$ hücredir. Zaman ve bellek $\Theta(nW)$'dir.

Ders sonrası notlar

- $P(3,3)$ **için önerilen ifade:** Hoca "$P(2,1)+v_3$ yazsam olur mu?" diye sordu, bir öğrenci evet dedi. 3. eşyanın ağırlığı 3 olduğu için doğru ifade $v_3 + P(2,3-3) = 20 + P(2,0) = 20$'dir. Bu, $P(2,3) = 22$ ile karşılaştırılır ve $P(3,3) = 22$ bulunur.
- **İlk sırt çantası örneği:** Derste yalnız ikili kombinasyonlar ve 5. eşya yazıldı. Tek eşyalı kombinasyonlar da ({1}, {2}, {3}, {4}) kapasiteye sığar; en büyüğü {4} = 50'dir. Sonuç değişmez: en iyi seçim {5} = 90 (Python ile kontrol edildi).
- **Para sırası formülü:** Bir öğrencinin önerdiği "$c_n - c_{n-2}$" ifadesi yanlıştı. Hoca bunu $\max\{c_n + F(n-2),\ F(n-1)\}$ olarak düzeltti; doğrusu budur.
- **Fibonacci'nin özyinelemeli karmaşıklığı:** Derste "çok fazla çağrı" dendi. Çağrı sayısı üstel büyür: $\Theta(\phi^n)$, $\phi \approx 1{,}618$. Dinamik programlama ile $\Theta(n)$ zamanda hesaplanır. Fibonacci için son iki değeri tutmak yeterli olduğundan bellek $\Theta(1)$'e indirilebilir.
- **Optimizasyon örneği:** "Grafı en az renkle boyama" bir optimizasyon problemidir. Ancak bu problem NP-zordur; dinamik programlama ile genel olarak polinom zamanda çözülemez. Örnek, yalnız "optimizasyon problemi nedir" sorusuna verilmiştir.
- **Sırt çantası karmaşıklığı:** $\Theta(nW)$ sözde polinomdur (pseudo-polynomial). W çok büyükse tablo da çok büyür. Hocanın "bellek ihtiyacını önceden hesaplayın" uyarısı bu durum için önemlidir.
- Tahta karelerindeki tablo değerleri ve iki örneğin sonuçları (17 ve 37) Python ile doğrulandı.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.